

Explain Delete and Earn DP

Yes. Let's build **Delete and Earn (LeetCode 740)** from the ground up. The important part is not memorizing the code — it is understanding **why the problem becomes House Robber**.

The uploaded notes use exactly this transformation: group equal values into total points, then solve a non-adjacent-selection DP.

1. First understand the actual problem

Suppose:

```
nums = [2, 2, 3, 3, 3, 4]
```

You can pick any number.

If you pick x :

-
-
-
-

This last point is extremely important.

Suppose we pick 3

Original:

```
[2, 2, 3, 3, 3, 4]
```

Pick one 3.

We earn:

```
3 points
```

Then:

```
3 - 1 = 2 → delete all 2s
3 + 1 = 4 → delete all 4s
```

So:

```
[2, 2, 3, 3, 3, 4]
      ↑
    picked
```

becomes:

```
[3, 3]
```

The other 3s are **NOT deleted**.

So we can take them too:

```
3 + 3 + 3 = 9
```

Therefore, if we decide to take the value 3, we should think:

```
TAKE 3 → get all the 3s → total = 9
```

That's why duplicates are combined.

2. Why do we combine duplicates?

This is the first major trick.

For:

```
[2, 2, 3, 3, 3, 4]
```

count each value:

Value	Frequency	Total points
2	2	$2 + 2 = 4$
3	3	$3 + 3 + 3 = 9$
4	1	4

So we create:

```
points[2] = 4
points[3] = 9
points[4] = 4
```

This is exactly the grouping described in your uploaded material.

Now instead of thinking:

```
2
2
3
3
3
4
```

we think:

```
value:  2  3  4
points: 4  9  4
```

Now the problem becomes much easier.

3. Why can we treat all duplicates as ONE decision?

This is something you specifically asked about.

Suppose:

```
nums = [3, 3, 3]
```

If we decide to take 3, taking only one 3 makes no sense.

Why?

Because picking 3 only destroys:

```
2 and 4
```

It does **not** destroy another 3.

Therefore, after deciding to take value 3, we can eventually collect:

```
3 + 3 + 3 = 9
```

So:

```
TAKE 3
```

means:

```
TAKE ALL 3s
```

and therefore:

```
points[3] = 9;
```

The original notes explicitly make this distinction: other copies of x remain available after choosing x .

4. Now comes the big observation

After grouping:

```
value:  1  2  3  4  5
points: 0  4  9  4  0
```

Look at what happens.

If we take 2:

```
2 → cannot take 1 or 3
```

If we take 3:

```
3 → cannot take 2 or 4
```

If we take 4:

```
4 → cannot take 3 or 5
```

So what do we have?

We're selecting values where **adjacent values cannot both be selected**.

That is exactly the structure of **House Robber**.

The notes describe this conversion as:

```
individual numbers
    ↓
total points for each number
    ↓
House Robber DP
```

5. Why do we take `maxi`?

Now your next question:

Why do we find the maximum element?

Suppose:

```
nums = [2, 2, 3, 3, 3, 4]
```

The largest number is:

```
maxi = 4
```

So our possible values are:

```
0 1 2 3 4
```

We create:

```
vector<int> points(maxi + 1, 0);
```

giving:

```
index:  0  1  2  3  4
points: 0  0  4  9  4
```

Why do we need `maxi`?

Because our DP state is based on the **value**, not the position in `nums`.

We want:

```
solve(1)
solve(2)
solve(3)
solve(4)
```

So we need an indexed structure where:

```
points[x]
```

means:

How many total points do I get if I choose value `x`?

For example:

```
points[3] = 9;
```

means:

If I choose number 3, I can earn 9 total points.

6. Why does recursion start from `maxi`?

This is another important concept.

We define:

```
solve(x)
```

as:

Maximum points we can earn using values from 1 to `x`.

Therefore, if the largest value is 4:

```
solve(4)
```

means:

```
Find the maximum answer considering values 1, 2, 3 and 4.
```

That's why we start:

```
solve(maxi)
```

We're asking the DP to consider the **entire range of possible values**.

The uploaded notes define `solve(x)` in exactly this way.

7. The Take / Skip logic

This is the heart of the problem.

At every value `x`, we have exactly two choices.

Choice 1: Skip `x`

Suppose we're at:

```
x = 4
```

and decide:

```
DON'T TAKE 4
```

Then nothing special happens.

We can continue considering:

```
1, 2, 3
```

So:

```
skip = solve(x - 1);
```

For 4:

```
skip = solve(3);
```

8. Choice 2: Take `x`

Now suppose:

```
TAKE 4
```

We get:

```
points[4]
```

But taking 4 destroys all 3s and 5s.

So we cannot take:

```
3
```

Therefore, after taking 4, the next safe value is:

```
2
```

That's why:

```
take = points[x] + solve(x - 2);
```

For `x = 4`:

```
take = points[4] + solve(2);
```

9. Why `x - 2`?

This is the part you specifically asked:

```
How does x-1 and x+1 become x-2?
```

Don't think that we are literally changing the original problem from:

```
x-1 and x+1
```

to:

```
x-2
```

That's not what's happening.
Instead, look at the DP direction.
We're processing values from:

```
1 → 2 → 3 → 4 → ...
```

Suppose we're currently deciding whether to take x .
If we take x , then:

```
x-1 → forbidden  
x   → taken  
x+1 → forbidden
```

Since we're moving **backward** in the recursion, the closest value we are allowed to consider is:

```
x - 2
```

Therefore:

```
TAKE x  
  ↓  
cannot use x-1  
  ↓  
jump to x-2
```

That's why:

```
points[x] + solve(x - 2)
```

The uploaded explanation states this exact Take/Skip transition.

10. The recurrence

Now everything comes together.
At every x :

Skip

```
solve(x - 1)
```

Take

```
points[x] + solve(x - 2)
```

We want the better option:

```
solve(x) = max(  
    solve(x - 1),  
    points[x] + solve(x - 2)  
);
```

This is exactly the House Robber recurrence.

11. Example completely

Take:

```
nums = [2,2,3,3,3,4]
```

We get:

```
points:  
  
index:  0  1  2  3  4  
        -----  
points: 0  0  4  9  4
```

Start:

```
solve(4)
```

At 4:

```
skip 4 = solve(3)
```

```
take 4 = 4 + solve(2)
```

Therefore:

```
solve(4) = max(solve(3), 4 + solve(2))
```

At 3:

```
skip 3 = solve(2)
```

```
take 3 = 9 + solve(1)
```

Therefore:

```
solve(3) = max(solve(2), 9 + solve(1))
```

At 2:

```
skip 2 = solve(1)
```

```
take 2 = 4 + solve(0)
```

Base:

```
solve(0) = 0
```

```
solve(1) = 0
```

Then:

```
solve(2) = max(0, 4)
          = 4
```

```
solve(3) = max(4, 9)
          = 9
```

```
solve(4) = max(9, 4 + 4)
          = 9
```

Answer:

```
9
```

We choose all the 3s.

12. Code 1 — Pure Recursion

First understand the raw recursion without DP.

```
class Solution {
public:
    int solve(int x, vector<int>& points) {
        if (x <= 0) return 0;

        int skip = solve(x - 1, points);
        int take = points[x] + solve(x - 2, points);

        return max(skip, take);
    }

    int deleteAndEarn(vector<int>& nums) {
        int maxi = 0;

        for (int x : nums) {
            maxi = max(maxi, x);
        }

        vector<int> points(maxi + 1, 0);

        for (int x : nums) {
```

```

        points[x] += x;
    }

    return solve(maxi, points);
}
};

```

Problem with this?

Huge repeated recursion.

For example:

```

solve(5)
├─ solve(4)
│   ├── solve(3)
│   └── solve(2)
└─ solve(3)
    ├── solve(2)
    └── solve(1)

```

`solve(3)` and `solve(2)` are calculated repeatedly.

That's why we use memoization.

13. Code 2 — Recursion + Memoization

```

class Solution {
public:
    int solve(int x, vector<int>& points, vector<int>& dp) {
        if (x <= 0) return 0;

        if (dp[x] != -1) return dp[x];

        int skip = solve(x - 1, points, dp);
        int take = points[x] + solve(x - 2, points, dp);

        return dp[x] = max(skip, take);
    }

    int deleteAndEarn(vector<int>& nums) {
        int maxi = 0;

        for (int x : nums) {
            maxi = max(maxi, x);
        }

        vector<int> points(maxi + 1, 0);

        for (int x : nums) {
            points[x] += x;
        }

        vector<int> dp(maxi + 1, -1);

        return solve(maxi, points, dp);
    }
};

```

The three lines you should be able to explain in an interview are:

```
points[x] += x;
```

Group duplicates.

```
int skip = solve(x - 1, points, dp);
```

Don't take `x`.

```
int take = points[x] + solve(x - 2, points, dp);
```

Take `x`, so `x-1` is unavailable.

14. Code 3 — Tabulation

Now let's remove recursion.

Our recurrence is:

```
dp[x] = max(dp[x-1], points[x] + dp[x-2])
```

Base:

```
dp[0] = 0
dp[1] = points[1]
```

Then:

```
class Solution {
public:
    int deleteAndEarn(vector<int>& nums) {
        int maxi = 0;

        for (int x : nums) {
            maxi = max(maxi, x);
        }

        vector<int> points(maxi + 1, 0);

        for (int x : nums) {
            points[x] += x;
        }

        vector<int> dp(maxi + 1, 0);

        dp[0] = 0;

        if (maxi >= 1) {
            dp[1] = points[1];
        }

        for (int x = 2; x <= maxi; x++) {
            int skip = dp[x - 1];
            int take = points[x] + dp[x - 2];

            dp[x] = max(skip, take);
        }

        return dp[maxi];
    }
};
```

The important conversion is:

Memoization

```
solve(x - 1)
solve(x - 2)
```

becomes:

Tabulation

```
dp[x - 1]
dp[x - 2]
```

And we calculate from small → large:

```
0 → 1 → 2 → 3 → 4 → ... → maxi
```

15. Code 4 — Space Optimization

Now look at the recurrence:

```
dp[x] = max(dp[x - 1], points[x] + dp[x - 2]);
```

To calculate current `dp[x]`, we only need:

```
dp[x-1]
dp[x-2]
```

We don't need the entire DP array.

So maintain two variables:

```
prev2 = dp[x-2]
prev1 = dp[x-1]
```

Then:

```
current = max(prev1, points[x] + prev2)
```

After that:

```
prev2 = prev1
prev1 = current
```

Final space-optimized code

```
class Solution {
public:
    int deleteAndEarn(vector<int>& nums) {
        int maxi = 0;

        for (int x : nums) {
            maxi = max(maxi, x);
        }

        vector<int> points(maxi + 1, 0);

        for (int x : nums) {
            points[x] += x;
        }

        int prev2 = 0;
        int prev1 = points[1];

        for (int x = 2; x <= maxi; x++) {
            int take = points[x] + prev2;
            int skip = prev1;

            int current = max(take, skip);

            prev2 = prev1;
            prev1 = current;
        }

        return prev1;
    }
};
```

There is one small edge case here: if `maxi == 0`, `points[1]` would be invalid. Since LeetCode's values are positive, this isn't normally an issue, but a robust version can avoid it:

```
class Solution {
public:
    int deleteAndEarn(vector<int>& nums) {
        int maxi = 0;

        for (int x : nums) {
            maxi = max(maxi, x);
        }

        vector<int> points(maxi + 1, 0);

        for (int x : nums) {
            points[x] += x;
        }

        int prev2 = 0;
        int prev1 = 0;

        for (int x = 1; x <= maxi; x++) {
```

```

        int take = points[x] + prev2;
        int skip = prev1;

        int current = max(take, skip);

        prev2 = prev1;
        prev1 = current;
    }

    return prev1;
}
};

```

This is the version I'd memorize.

16. The entire pattern in one picture

Original array

[2, 2, 3, 3, 3, 4]
 ↓
 group equal values

↓
 points[2] = 4
 points[3] = 9
 points[4] = 4
 ↓

[0, 0, 4, 9, 4]
 ↓

Think of each VALUE as a house

value:	1	2	3	4
points:	0	4	9	4

↓
 can't take adjacent
 ↓
 TAKE / SKIP DP

And the mental formula is:

SKIP x
 ↓
 solve(x-1)

TAKE x
 ↓
 points[x]
 +
 solve(x-2)

Therefore:

```

solve(x) = max(
    solve(x - 1),
    points[x] + solve(x - 2)
);

```

That is the entire problem.

Don't memorize "Delete and Earn = House Robber." Memorize the reasoning:

Same values → combine their total earnings.
Taking x blocks adjacent values $x-1$ and $x+1$.
Therefore choose non-adjacent values → Take/Skip DP.